



CHAPTER 5: JAVA DATABASE PROGRAMMING

CS 313: ADVANCED PROGRAMMING LANGUAGE

OUTLINE

- Relational Database Systems
- SQL
- JDBC
- PreparedStatement

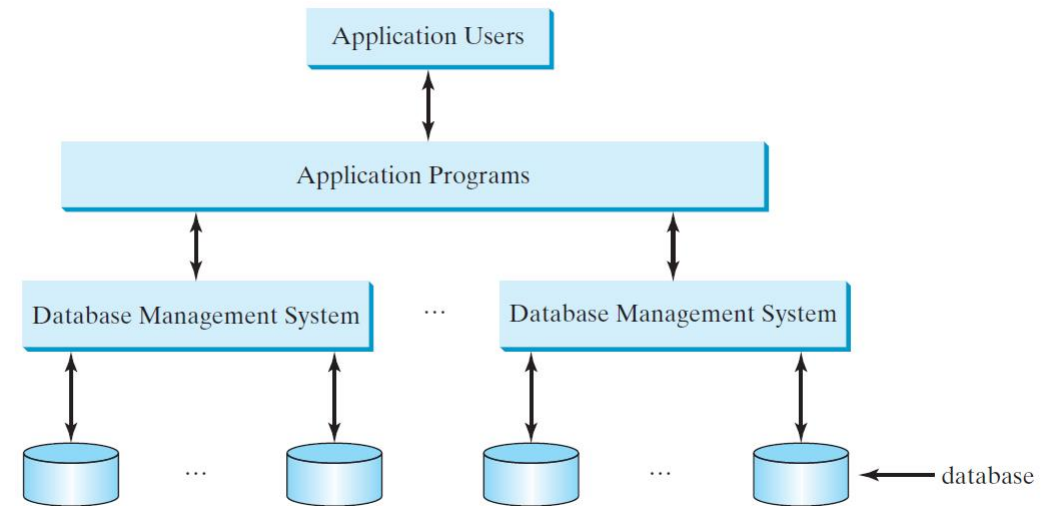
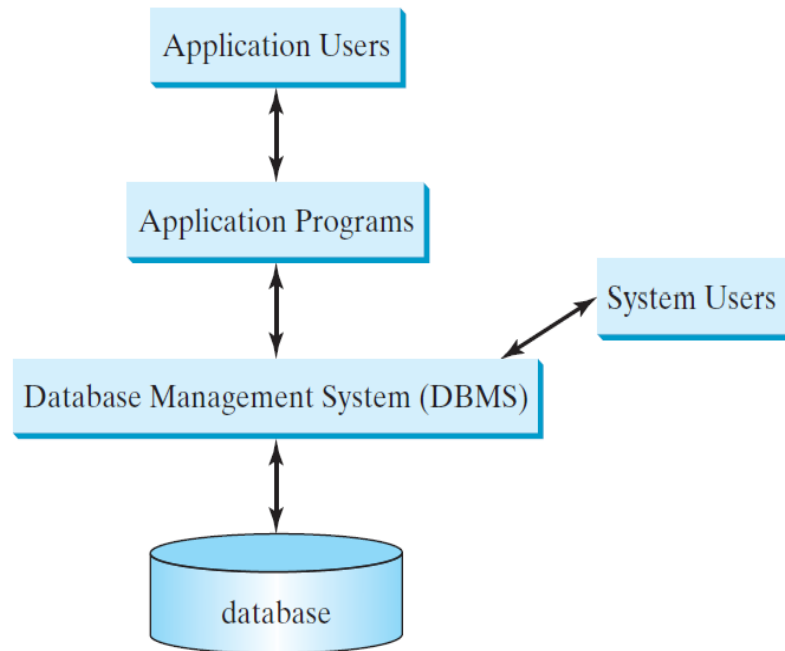
RELATIONAL DATABASE SYSTEMS

Definition & Architecture

- Database system consists of a **database**, the software that stores and manages data in the database, and the application programs that present data and enable the user to interact with the database system,
- A database is a repository of data that form information (MySQL, Oracle, IBM's DB2 and Informix, Microsoft SQL Server, or Sybase). When you purchase the software it comprises a **database management system (DBMS)**.
- Application programs are built on top of the DBMS for customers to access and update the database. Application programs can be viewed as the interfaces between the database system and its users.
- Application programs may be stand-alone GUI applications or Web applications and may access several different database systems in the networks a repository

RELATIONAL DATABASE SYSTEMS

Definition & Architecture



RELATIONAL DATABASE SYSTEMS

Relational Structures

- SQL is the standard database language for defining and accessing databases.
- The relational model is built around a simple and natural structure. A ***relation*** is actually a table that consists of nonduplicate **rows**.
- Tables are easy to understand and use. The relational model provides a simple yet powerful way to represent data.
- A **row** of a table represents a ***record***, and a **column** of a table represents the ***value of a single***
- ***attribute*** of the record.
- In relational database theory, a row is called a ***tuple*** and a column is called an ***attribute***.

RELATIONAL DATABASE SYSTEMS

Relational Structures

Diagram illustrating the structure of the **Course Table**:

Relation/Table Name: **Course Table**

Columns/Attributes: **courseId**, **subjectId**, **courseNumber**, **title**, **numOfCredits**

Tuples/Rows:

courseId	subjectId	courseNumber	title	numOfCredits
11111	CSCI	1301	Introduction to Java I	4
11112	CSCI	1302	Introduction to Java II	3
11113	CSCI	3720	Database Systems	3
11114	CSCI	4750	Rapid Java Application	3
11115	MATH	2750	Calculus I	5
11116	MATH	3750	Calculus II	5
11117	EDUC	1111	Reading	3
11118	TTEC	1344	Database Administration	3

Student Table

ssn	firstName	mi	lastName	phone	birthDate	street	zipCode	deptID
444111110	Jacob	R	Smith	9129219434	1985-04-09	99 Kingston Street	31435	BIOL
444111111	John	K	Stevenson	9129219434	null	100 Main Street	31411	BIOL
444111112	George	K	Smith	9129213454	1974-10-10	1200 Abercorn St.	31419	CS
444111113	Frank	E	Jones	9125919434	1970-09-09	100 Main Street	31411	BIOL
444111114	Jean	K	Smith	9129219434	1970-02-09	100 Main Street	31411	CHEM
444111115	Josh	R	Woo	7075989434	1970-02-09	555 Franklin St.	31411	CHEM
444111116	Josh	R	Smith	9129219434	1973-02-09	100 Main Street	31411	BIOL
444111117	Joy	P	Kennedy	9129229434	1974-03-19	103 Bay Street	31412	CS
444111118	Toni	R	Peterson	9129229434	1964-04-29	103 Bay Street	31412	MATH
444111119	Patrick	R	Stoneman	9129229434	1969-04-29	101 Washington St.	31435	MATH
444111120	Rick	R	Carter	9125919434	1986-04-09	19 West Ford St.	31411	BIOL

Enrollment Table

ssn	courseId	dateRegistered	grade
444111110	11111	2004-03-19	A
444111110	11112	2004-03-19	B
444111110	11113	2004-03-19	C
444111111	11111	2004-03-19	D
444111111	11112	2004-03-19	F
444111111	11113	2004-03-19	A
444111112	11114	2004-03-19	B
444111112	11115	2004-03-19	C
444111112	11116	2004-03-19	D
444111113	11111	2004-03-19	A
444111113	11113	2004-03-19	A
444111114	11115	2004-03-19	B
444111115	11115	2004-03-19	F
444111115	11116	2004-03-19	F
444111116	11111	2004-03-19	D
444111117	11111	2004-03-19	D
444111118	11111	2004-03-19	A
444111118	11112	2004-03-19	D
444111118	11113	2004-03-19	B

The **Course** table and the **Enrollment** table are related through their common attribute **courseId**, and the **Enrollment** table and the **Student** table are related through **ssn**.

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

- **An *integrity constraint*** imposes a condition that all the legal values in a table must satisfy
- Three types of constraints:
 - **domain constraints**
 - **primary key constraints**
 - **foreign key constraints**
- **Domain constraints** and **primary key constraints** are known as ***intra-relational constraints***: a constraint involves only one relation.
- The **foreign key constraint** is ***inter-relational constraints***: constraint involves more than one relation.

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

1. Domain Constraints

- *Domain constraints* specify the permissible values for an attribute.
- Domains can be specified using standard data types, such as integers, floating-point numbers, fixed-length strings, and variant-length strings. The standard data type specifies a broad range of values
- Additional constraints can be specified to narrow the ranges.
 - **Example1:** specify that the **numOfCredits** attribute (in the **Course** table) must be greater than 0 and less than 5.
 - **Example2:** specify whether an attribute can be **null**, which is a special value in a database meaning unknown or not applicable. As shown in the **Student** table, **birthDate** may be **null**.

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

2. Primary Key Constraints

- To understand *primary keys*, it is helpful to know **superkeys, keys, and candidate keys**.
- A **superkey** is an attribute or a set of attributes that uniquely identifies the relation (i.e., no two tuples have the same values on a **superkey**).
- By definition, a relation consists of a set of distinct tuples. The set of all attributes in the relation forms a **superkey**.
- A key **K** is a minimal superkey, meaning that any proper subset of **K** is not a superkey.
- A relation can have several keys. In this case, each of the keys is called a **candidate key**.
- The *primary key* is one of the candidate keys designated by the database designer.
- The primary key is often used to identify tuples in a relation (e.g., **courseId** is the primary key in the **Course** table)

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

3. Foreign Key Constraints

- In a *relational database*, data are related. Tuples in a relation are related, and tuples in different relations are related through their **common attributes**.
- The common attributes are **foreign keys**.
- The *foreign key constraints* define the relationships among relations.
- set of attributes FK is a *foreign key* in a relation R that references relation T if it satisfies the following two rules:
 - The attributes in FK have the same domain as the primary key in T .
 - A nonnull value on FK in R must match a primary key value in T .

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

Enrollment Table

ssn	courseId	dateRegistered	grade
444111110	11111	2004-03-19	A
444111110	11112	2004-03-19	B
444111110	11113	2004-03-19	C
...			

Course Table

courseId	subjectId	courseNumber	title	numOfCredits
11111	CSCI	1301	Introduction to Java I	4
11112	CSCI	1302	Introduction to Java II	3
11113	CSCI	3720	Database Systems	3
...				

Each value in **courseId** in the Enrollment table must match a value in **courseId** in the Course table

Each row must have a value for **courseId**, and the value must be unique

Each value in the **numOfCredits** column must be greater than 0 and less than 5

RELATIONAL DATABASE SYSTEMS

Integrity Constraints

Enforcing Integrity Constraints

- The database management system enforces integrity constraints and rejects operations that would violate them.

Examples:

- Any Attempt to insert the new record (“11115,” “CSCI,” “2490,” “C++ Programming,” 0) into the **Course** table, it would fail because the credit hours must be greater than 0;
 - Any attempt to insert a record with the same primary key as an existing record in the table, the DBMS would report an error and reject the operation;
 - Any Attempt to delete a record from the **Course** table whose primary key value is referenced by the records in the **Enrollment** table, the DBMS would reject this operation.
- ~~✗~~ All relational database systems support primary key constraints and foreign key constraints but not all database systems support domain constraints.**

RELATIONAL DATABASE SYSTEMS

SQL

- Structured Query Language (SQL) is the language for defining tables and integrity constraints and for accessing and manipulating data.
- Application programs may allow users to access a database without directly using SQL, but these applications themselves must use SQL to access the database.
- SQL can be used on MySQL, Oracle, Sybase, IBM DB2, IBM Informix, MS Access, or any other relational database system

Creating and Dropping Tables

- Tables are the essential objects in a database. To create a table, use the create table statement to specify a table name, attributes, and types

```
create table Course (  
  courseId char(5),  
  subjectId char(4) not null,  
  courseNumber integer,  
  title varchar(50) not null,  
  numOfCredits integer,  
  primary key (courseId)  
);
```

```
create table Student (  
  ssn char(9),  
  firstName varchar(25),  
  mi char(1),  
  lastName varchar(25),  
  birthDate date,  
  street varchar(25),  
  phone char(11),  
  zipCode char(5),  
  deptId char(4),  
  primary key (ssn)  
);
```

```
create table Enrollment (  
  ssn char(9),  
  courseId char(5),  
  dateRegistered date,  
  grade char(1),  
  primary key (ssn, courseId),  
  foreign key (ssn) references  
    Student(ssn),  
  foreign key (courseId) references  
    Course(courseId)  
);
```

RELATIONAL DATABASE SYSTEMS

SQL

Simple Insert, Update, and Delete

Once a table is created, you can insert data into it. You can also update and delete records.

Insert

Syntax

```
insert into tableName [(column1, column2, ..., column)]  
values (value1, value2, ..., valuen);
```

Example

```
insert into Course (courseId, subjectId, courseNumber, title, numOfCredits)  
values ('11113', 'CSCI', '3720', 'Database Systems', 3);
```

Update

Syntax

```
update tableName  
set column1 = newValue1 [, column2 = newValue2, ...]  
[where condition];
```

Example

```
update Course  
set numOfCredits = 4  
where title = 'Database Systems';
```

RELATIONAL DATABASE SYSTEMS

SQL

Delete

Syntax

```
delete from tableName  
[where condition];
```

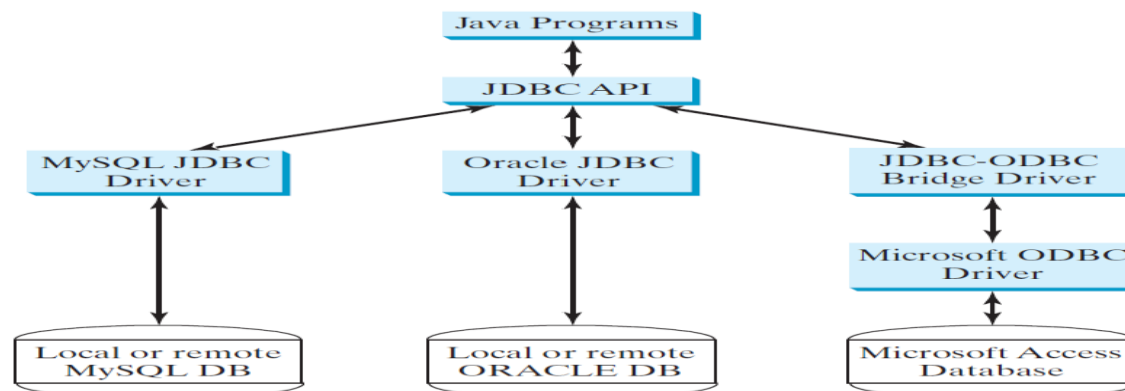
Example

```
delete from Course  
where title = 'Database Systems';
```

✗ SQL keywords are **NOT** case sensitive

JDBC

- JDBC is the Java API for accessing relational database.
- JDBC is not an acronym, but it is often thought to stand for **Java Database Connectivity**
- JDBC provides Java programmers with a uniform interface for accessing and manipulating relational databases.
- Using the JDBC API, applications written in the Java programming language can execute SQL statements, retrieve results, present data in a user-friendly interface
- The JDBC API can also be used to interact with multiple data sources in a distributed environment
- Java programs access and manipulate databases through **JDBC drivers**.



JDBC

- The JDBC API is a Java application program interface to generic SQL databases that enables Java developers to develop DBMS-independent Java applications using a uniform interface
- Four key interfaces are needed to develop any database application using Java: **Driver**, **Connection**, **Statement**, and **ResultSet**. The application:
 - application loads an appropriate driver using the **Driver** interface,
 - connects to the database using the **Connection**
 - creates and executes SQL statements using the **Statement** interface, and
 - Processes the result using the **ResultSet** interface if the statements return results

JDBC

Developing Database Applications using JDBC

1. Loading drivers

- An appropriate driver must be loaded using the statement shown below before connecting to a database. `Class.forName("JDBCDriverClass");`

Database	Driver Class
Access	<code>sun.jdbc.odbc.JdbcOdbcDriver</code>
MySQL	<code>com.mysql.jdbc.Driver</code>
Oracle	<code>oracle.jdbc.driver.OracleDriver</code>

- ✗ If Oracle drivers will be used , you have to add **mysql-connector-java-VERSION.jar** and **ojdbcVERSION.jar** in the classpath using the following DOS command on Windows:
set classpath=%classpath%;c:\book\lib\mysql-connector-java-VERSION.jar; c:\book\lib\ojdbcVERSION.jar
- ✗ If you use an IDE such as NetBeans, you need to **add these jar files into the library in the IDE.**

JDBC

Developing Database Applications using JDBC

1. Establishing the connection

- To connect to a database, use the static method `getConnection(databaseURL)` in the **DriverManager** class, as follows: `Connection connection = DriverManager.getConnection(databaseURL) ;` where `databaseURL` is the unique identifier of the database on the Internet.

Database	URL Pattern
Access	<code>jdbc:odbc:datasource</code>
MySQL	<code>jdbc:mysql://hostname/dbname</code>
Oracle	<code>jdbc:oracle:thin:@hostname:port#:oracleDBSID</code>

- The `databaseURL` for an Oracle database specifies the *hostname*, the *port#* where the database listens for incoming connection requests, and the *oracleDBSID* database name to locate a database.
Example: creates a `Connection` object for the Oracle database on `liang.armstrong.edu` with the username `scott` and password `tiger`:

```
Connection connection = DriverManager.getConnection ("jdbc:oracle:thin:@liang.armstrong.edu:1521:orcl", "scott", "tiger");
```

JDBC

Developing Database Applications using JDBC

3. Creating statements

- object of **Statement** can be viewed as a cart that delivers SQL statements for execution by the database and brings the result back to the program.
- Once a **Connection** object is created, you can create statements for executing SQL statements as follows:

```
Statement statement = connection.createStatement();
```

4. Executing statements

- SQL data definition language (DDL) and update statements can be executed using **executeUpdate(String sql)**
- an SQL query statement can be executed using **executeQuery(String sql)**. The result of the query is returned in **ResultSet**

Example1: executes the SQL statement **create table Temp (col1 char(5), col2 char(5)):**

```
statement.executeUpdate ("create table Temp (col1 char(5), col2 char(5))");
```

Example2: executes the SQL query **select firstName, mi, lastName from Student where lastName = 'Smith':**

// Select the columns from the Student table

```
ResultSet resultSet = statement.executeQuery("select firstName, mi, lastName from Student where lastName='Smith'");
```

Developing Database Applications using JDBC

4. Processing ResultSet.

- The **ResultSet** maintains a table whose current row can be retrieved. The initial row position is **null**.
- You can use the **next** method to move to the next row and the various getter methods to retrieve values from a current row.

Example : displays all the results from the preceding SQL query.

// Iterate through the result and print the student names

while (resultSet.next())

**System.out.println(resultSet.getString(1) + " " + resultSet.getString(2) + " " +
resultSet.getString(3))**

- The **getString(1)**, **getString(2)**, and **getString(3)** methods retrieve the column values for **firstName**, **mi**, and **lastName**, respectively.
- Alternatively, you can use **getString("firstName")**, **getString("mi")**, and **getString("lastName")** to retrieve the same three column values

PREPARED STATEMENT

- Once a connection to a particular database is established, it can be used to send SQL statements from your program to the database.
 - The **Statement** interface is used to execute static SQL statements that **don't contain any parameters**.
 - The **PreparedStatement** interface, extending **Statement**, is used to execute a precompiled SQL statement **with or without parameters**.
- **PreparedStatement enables you to create parameterized SQL statements**
- A **PreparedStatement** object is created using the **prepareStatement** method in the **Connection** interface.
- **Example:** creates a PreparedStatement for an SQL insert statement:

```
PreparedStatement preparedStatement = connection.prepareStatement ("insert into Student (firstName, mi, lastName) values (?, ?, ?)");
```

- This **insert** statement has three question marks as placeholders for parameters representing values for **firstName**, **mi**, and **lastName** in a record of the **Student** table.
- the **PreparedStatement** interface inherits all the methods defined in **Statement**. It also provides the methods for setting parameters in the object of **PreparedStatement**. These methods are used to set the values for the parameters before executing statements or procedures. In general, the setter methods have the following name and signature:

```
setX(int parameterIndex, X value);
```

where *X* is the type of the parameter, and **parameterIndex** is the index of the parameter in the statement. The index starts from **1**. For example, the method **setString(int parameterIndex, String value)** sets a **String** value to the specified parameter

PREPARED STATEMENT

Example:

```
PreparedStatement preparedStatement = connection.prepareStatement ("insert into Student (firstName, mi, lastName)  
values (?, ?, ?)");
```

```
preparedStatement.setString(1, "Jack");
```

```
preparedStatement.setString(2, "A");
```

```
preparedStatement.setString(3, "Ryan");
```

»»»Pass the parameters "**Jack**", "**A**", and "**Ryan**" to the placeholders for **firstName**, **mi**, and **lastName** in **preparedStatement**

- After setting the parameters, prepared statement can be executed by invoking **executeQuery()** for a SELECT statement and **executeUpdate()** for a DDL or update statement.
- The **executeQuery()** and **executeUpdate()** methods are similar to the ones defined in the **Statement** interface except that they don't have any parameters, because the SQL statements are already specified in the **prepareStatement** method when the object of **PreparedStatement** is created.

SUMMARY

- Concepts of *database systems, relational databases, relational data models, data integrity*, and *SQL*. have been introduced
- The Java API for developing Java database applications is called *JDBC*. JDBC provides Java programmers with a uniform interface for accessing and manipulating relational databases.
- The JDBC API consists of classes and interfaces for establishing connections with databases, sending SQL statements to databases, processing the results of SQL statements
- Since a JDBC driver serves as the interface to facilitate communications between JDBC and a proprietary database, JDBC drivers are database specific.
- Four key interfaces are needed to develop any database application using Java: **Driver**, **Connection**, **Statement**, and **ResultSet**. These interfaces define a framework for generic SQL database access. The JDBC driver vendors provide implementation for them.
- A JDBC application loads an appropriate driver using the **Driver** interface, connects to the database using the **Connection** interface, creates and executes SQL statements using the **Statement** interface, and processes the result using the **ResultSet** interface if the statements return results.
- The **PreparedStatement** interface is designed to execute dynamic SQL statements with parameters. These SQL statements are precompiled for efficient use when

repeatedly executed.

dr. IMF, 2024

That's All

- **Textbook : Intro to Java Programming, Comprehensive Version (10th Edition), Y. Daniel Liang: Chapter 32**